

ML-based Anomaly Detection in Cyber Physical Systems

Ventrapragada Krishna Sameer

Guide: Prof. Chester Rebeiro

Abstract

Cyber-Physical Systems (CPS) integrate computation, communication, and physical processes, and are widely used in industrial environments through Industrial Control Systems (ICS). Many such systems rely on the Modbus protocol, which lacks strong built-in security features and is therefore vulnerable to cyber threats. This project explores the use of Machine Learning (ML) techniques for anomaly detection in Modbus network traffic using a publicly available dataset. Multiple ML models were evaluated to learn normal communication patterns and identify deviations that may indicate malicious activity. Model performance was assessed using metrics such as accuracy, precision, recall, and F1-score. The results show that ML-based methods can effectively detect anomalous Modbus traffic and enhance security monitoring in CPS environments. The study also highlights the importance of proper preprocessing, feature selection, and threshold tuning for reliable detection.

Contents

Abstract	i
1 Introduction	2
1.1 Motivation	2
1.2 CIC Modbus Dataset 2023	2
2 Analysing existing work	4
3 Our work	5
3.1 Feature Extraction	5
3.2 Long Short Term Memory	5
3.3 Kit-NET Architecture	6
3.4 Tuning the model	7
4 Conclusion	8
4.1 Results	8
4.2 Future work	8
5 Acknowledgements	10

Introduction

1.1 Motivation

Cyber-Physical Systems (CPS) integrate computation, communication networks, and physical processes to monitor and control real-world operations. A major class of CPS is Industrial Control Systems (ICS), which are used in sectors such as power systems, manufacturing, water treatment, etc. These systems rely on sensors, actuators, controllers, and communication networks to ensure safe and efficient industrial operation.

Common ICS architectures include Supervisory Control and Data Acquisition (SCADA) systems, which provide centralized monitoring, data collection, alarm handling, and remote control of industrial processes. Modern industrial environments also use Intelligent Electronic Devices (IEDs), which are microprocessor-based devices capable of protection, measurement, automation, and control, especially in power and substation networks. Communication among these components is often carried out using industrial protocols such as Modbus. Modbus is a widely used communication protocol designed for exchanging data between controllers, sensors, HMIs, and SCADA systems over serial links or Ethernet networks (Modbus TCP). Its simplicity and interoperability have made it popular in industrial automation.

However, Modbus was not originally designed with strong security features. It lacks mechanisms such as authentication and encryption, making industrial networks vulnerable to attacks such as unauthorized access, spoofing, replay attacks, and denial-of-service attempts.

To address these threats, Machine Learning (ML) based anomaly detection has gained attention. Instead of relying only on known attack signatures, ML models learn normal traffic behavior and identify unusual patterns. In this project, ML techniques are applied to a Modbus dataset to detect anomalous network activity and improve the security of CPS environments. The code and results related to the work are available at <https://github.com/SameerCS23iitm/ugrc>.

1.2 CIC Modbus Dataset 2023

For this project, the CIC Modbus dataset, Canadian Institute for Cybersecurity 2023; Boakye-Boateng, Ghorbani, and Lashkari 2024 was used as the primary benchmark dataset. It was developed by the Canadian Institute for Cybersecurity (CIC) for research on intrusion detection and anomaly detection in Modbus-based Industrial Control Systems. The dataset contains labeled network traffic captures (PCAP files) and attack logs collected from a simulated substation environment.

The testbed was implemented using a Docker-based simulated network representing industrial devices such as SCADA HMIs and Intelligent Electronic Devices (IEDs). Python scripts were used to emulate the operational logic of these devices. In the setup, IEDs periodically changed voltage values or responded to SCADA requests, while the SCADA system performed control actions such as tap-changing and opening or closing switches based on voltage conditions. The setup used for extracting the dataset is shown in Figure 1.1.

Traffic was collected under both normal and attack conditions. The benign portion contains legitimate

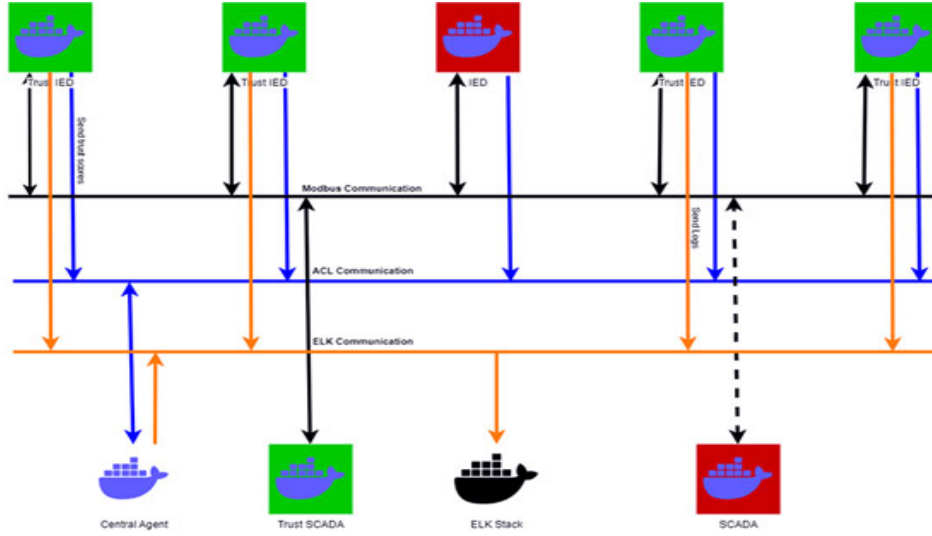


Figure 1.1: Source: Canadian Institute for Cybersecurity 2023

Modbus communication between the simulated devices. The attack portion provides three scenarios: Compromised SCADA, Compromised IED and External Attacker, which includes several realistic Modbus attack scenarios such as reconnaissance, query flooding, false data injection, delay response, modify length parameters, stacking Modbus frames, brute-force write attacks, and baseline replay attacks. Many of these scenarios were designed using techniques inspired by the MITRE ICS ATTACK framework. The effects of some of these attacks are specified in Table 1.1.

The dataset is valuable because it provides recent, protocol-specific traffic for Modbus security research instead of generic enterprise network traffic. In this project, the CIC Modbus Dataset 2023 was used for preprocessing, feature extraction, model training, and evaluation of Machine Learning based anomaly detection methods. However, this dataset does not provide process-level values provided by more popular testbeds such as SWaT and WADI.

Table 1.1: Attack Types in CIC Modbus Dataset 2023

Attack Type	Description
Reconnaissance	Scanning the network to identify active devices, slave IDs, registers, and supported functions.
Query Flooding	Sending excessive Modbus queries to overload the device or communication channel.
False Data Injection	Injecting forged values or responses to mislead monitoring or control systems.
Delay Response	Intentionally delaying valid responses, causing timeouts or stale data.
Modify Length Parameters	Tampering packet length fields to create malformed Modbus frames.
Stacking Frames	Combining multiple Modbus frames abnormally in a single transmission.
Brute-force Write Attack	Repeatedly issuing write commands to force unauthorized register or coil changes.
Baseline Replay Attack	Replaying previously captured valid traffic to mimic legitimate communication.

Analysing existing work

Most of the existing work in anomaly detection for Cyber-Physical and Industrial Control Systems relies on process-level data such as sensor readings, actuator states, voltage, pressure, temperature, or flow values. These features directly reflect the physical behavior of the system and often make abnormal conditions easier to detect. However, working with a Modbus network dataset is more challenging, since it primarily contains communication traffic and protocol interactions rather than explicit process measurements. As a result, we were restricted to using communication-layer values. However, we reviewed several existing research papers on ICS anomaly detection and Modbus security as references to understand the state-of-the-art works on the topic.

Sekaran et al. extracted packet-level features such as packet-count, byte-count and ran several classic ML models such as Decision Tree, Random Forest, K-means, etc., but they provided only the accuracies and did not elaborate on features like precision, recall and f1-score, which are more relevant in imbalanced datasets such as ours.

Abdelaty, Doriguzzi-Corin, and Siracusa used the SWaT and WADI datasets for training. Time-series data of sensor and actuator values were taken as input. For feature extraction and mapping, two branches: A wide network and a deep CNN were used. The combined results were then fed into another neural network to predict the values. The main target of this model is to adapt to the progressive changes in the normal behavior. This paper differs from our work in terms of what data is available for training, since malicious activator and sensor values are more apparent than network flow-values.

Almendli and Mohasefi is yet another paper that used the SWaT dataset. In addition to actuator and sensor values, pcap readings were also used. An encoding of these values, called Modbus-values, was used, which we were unable to extract.

Mirsky et al. uses an ensemble of autoencoders for intrusion detection. The main aim of the paper is to make the process light weight so that it can be deployed without holding too much data at a given point of time. They use damped statistics instead of a sliding window for memory efficiency. This was the paper that formed the basis of our work in the later phases.

Our work

3.1 Feature Extraction

The feature extraction process began with an analysis of the attack scenarios present in the CIC Modbus Dataset 2023 and the network behaviors expected during each attack. Certain attacks such as query flooding or reconnaissance are likely to create visible changes in traffic volume, destination diversity, or request frequency. Similarly, brute-force write attacks may increase the number of write commands, while malformed-packet attacks such as length modification or stacked Modbus frames may produce protocol inconsistencies. This motivated the use of packet statistics, Modbus function code usage, etc. as candidate features.

We analyzed the dataset for some of these features to see if there were noticeable differences in some scenarios. However, only scenarios such as query flooding, as shown in Figure 3.1, had visible changes. Attacks like baseline replay and time delay can easily be mistaken for normal traffic. Since we were restricted to packets alone, we decided to choose the features that are most likely to be affected in attack scenarios. The main features that we took into consideration are listed in Table 3.1. We then ran our models on two of the provided scenarios: External Attacker and Compromised SCADA.

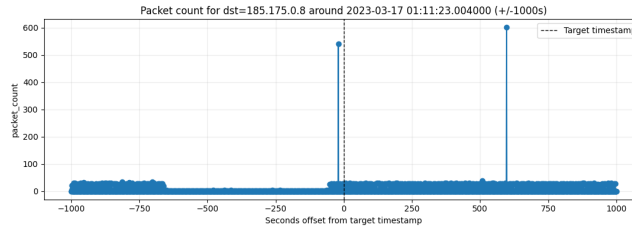


Figure 3.1: Noticeable spike in packet count near the attack.

Table 3.1: Some Key Features Extracted (per-IP, per-second)

Feature	Purpose / Relevance
Packet Count (Tx/Rx)	Detects flooding, scanning, or sudden bursts in traffic volume.
Bytes Sent / Received	Identifies abnormal communication load or large payload transfers.
Inter-arrival Time Mean	Useful for detecting replay, flooding, or delayed responses.
Write Request Count	High values may indicate unauthorized control or brute-force writes.
Function Code Distribution	Unusual command usage helps detect malicious Modbus activity.

3.2 Long Short Term Memory

Our first approach for anomaly detection was based on a Long Short-Term Memory (LSTM) network. Since network traffic is sequential in nature and attacks may depend on temporal behavior over time, LSTMs appeared to be a suitable starting point. The extracted per-IP, per-second features were arranged as time-series sequences and used to train a baseline LSTM model for binary classification

of benign and malicious traffic. This was run on the compromised SCADA dataset in our case since the other scenario’s distribution was not very helpful for training.

However, during experimentation, it was observed that the dataset suffered from significant class imbalance, with benign traffic greatly outnumbering attack samples. As a result, the baseline LSTM failed to learn meaningful attack patterns and converged to predicting nearly all samples as benign. Although this produced superficially high accuracy, the model showed poor recall and failed to detect anomalies effectively.

To improve learning, we next explored feature weighting and importance-guided preprocessing. Tree-based models such as XGBoost and Random Forest were trained to estimate feature importance scores, while an Autoencoder was also used to identify features contributing more strongly to reconstruction error. These importance measures were then used to reweigh the features before retraining the LSTM model.

Despite these modifications, the LSTM still did not produce satisfactory results. Detection performance remained inconsistent, especially for minority attack classes, indicating that sequence modeling alone was not sufficient for the given feature representation since the flow level values were very similar. Based on these observations, we moved toward an Autoencoder-based anomaly detection framework with a sequential touch, which is discussed in the next section.

3.3 Kit-NET Architecture

Kitsune, by Mirsky et al. 2018, is a lightweight network intrusion detection framework that uses an ensemble of small autoencoders to learn normal traffic behavior and detect anomalies through reconstruction error. Its design is especially suitable for network traffic, where relationships between subsets of features can be learned efficiently without requiring large labeled attack datasets.

The core idea of Kitsune is to first group related input features into smaller clusters and assign each cluster to a dedicated autoencoder. Each autoencoder learns to reconstruct only its assigned feature subset, producing a reconstruction error for that group. These errors are then combined and passed to a final output autoencoder, which learns the normal pattern of reconstruction errors across all groups. During inference, abnormal traffic produces higher reconstruction error, allowing anomalies to be detected. A high-level overview of the architecture is provided by Figure 3.2.

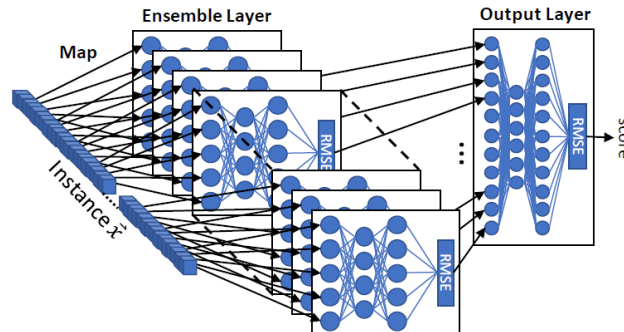


Figure 3.2: Source: Mirsky et al. 2018 Fig.1

In its original form, Kitsune operates on online packet streams, where features are extracted from packets in real time and processed sequentially. However, our dataset was available as a large offline traffic capture rather than a live stream. Therefore, instead of applying Kitsune directly to raw

packets, we used the per-IP, per-second aggregated features obtained from the previous stage. To preserve the original design principles, we adopted the Kit-NET architecture based on the publicly available open-source implementation, Mirsky 2017, and adapted its input pipeline for our feature set. This allowed us to retain the core strengths of Kitsune while making it suitable for our use-case, although the aggregation process may slightly smooth or dampen short-lived anomalies. This model was trained on purely benign data, to learn a reduced form, and was then run on both of the test scenarios so that the anomalies can be detected using the reconstruction errors.

3.4 Tuning the model

After adapting the KitNET architecture to the aggregated Modbus feature set, the next step was to tune the model hyperparameters to obtain better anomaly detection performance. Since KitNET relies on an ensemble of small autoencoders followed by a final output autoencoder, its behavior is strongly influenced by how input features are grouped and how quickly the autoencoders learn normal patterns.

One of the primary parameters explored was the maximum number of features assigned to each cluster autoencoder. Smaller cluster sizes allow each autoencoder to learn simpler and more specialized feature relationships, while larger cluster sizes capture broader interactions among multiple traffic features. Different values were tested to study the trade-off between local feature specialization and overall representation capacity.

Another important parameter was the learning rate used during training. A high learning rate may lead to unstable convergence, while a very low learning rate can slow learning and reduce responsiveness. Multiple learning rates were therefore evaluated to obtain more stable reconstruction behavior.

Through experimentation with different combinations of cluster sizes and learning rates, the model showed moderate improvement in separating benign and anomalous traffic compared to the initial baseline configuration. While the performance was not strong enough to be considered a highly reliable classifier, these tuning steps helped improve consistency over the previous attempt. The performance of the proposed model under different attack scenarios is summarized in Table 3.2.

Table 3.2: Kit-NET Hyperparameter Tuning Results under Different Attack Scenarios

Scenario: External Attacker						
Cluster Size	Hidden Ratio	Learning Rate	Accuracy	Precision	Recall	F1-score
8	0.75	0.10	0.9446	0.1195	0.9700	0.2127
8	0.50	0.08	0.9687	0.1932	0.9600	0.3216
16	0.75	0.10	0.9270	0.0949	0.9900	0.1732
16	0.50	0.08	0.8729	0.0568	0.9900	0.1074
Scenario: Compromised SCADA						
Cluster Size	Hidden Ratio	Learning Rate	Accuracy	Precision	Recall	F1-score
8	0.75	0.10	0.8646	0.0347	0.1442	0.0560
8	0.50	0.08	0.8654	0.0375	0.1553	0.0604
16	0.75	0.10	0.8850	0.0373	0.1262	0.0576
16	0.50	0.08	0.8810	0.0358	0.1262	0.0557

Conclusion

4.1 Results

In the external attacker scenario, the model demonstrates a strong ability to detect anomalous activity, as reflected by the consistently high recall values. This indicates that most attack instances are successfully identified. However, this comes at the cost of relatively low precision, suggesting the presence of false positives where benign traffic is misclassified as anomalous. Consequently, the F1-score remains moderate despite high detection capability. In safety-critical environments such as power grids, however, high recall is prioritized over precision, as the cost of an undetected attack far exceeds that of a false alarm.

In contrast, the model performs rather poorly in the compromised SCADA scenario. Both precision and recall values are significantly lower, indicating difficulty in distinguishing malicious behavior when attacks originate from within the system. This confirms that internal attacks exhibit greater similarity to normal communication patterns, making them inherently harder to detect using traffic-based features alone. Confusion matrices for both scenarios are shown below to illustrate the distribution of true and false predictions.

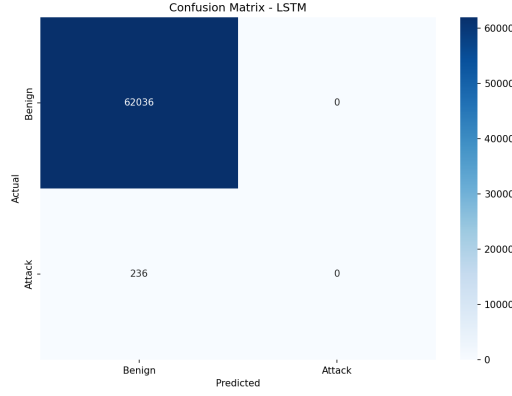
Despite these limitations, the Kitsune-based approach shows a clear improvement over the earlier LSTM model, which largely failed to learn meaningful patterns due to class imbalance and tended to classify nearly all samples as benign. The shift to an autoencoder-based framework enabled better capture of deviations from normal behavior, even though the overall performance is still not strong enough for reliable deployment. These observations highlight both the strengths and limitations of the current approach and motivate further refinements, which are discussed in the subsequent section on future work. The results of our experiments are presented as confusion matrices in Figure 4.1. Benchmarking results are not reported, as the model has not yet reached sufficient performance to meaningfully evaluate efficiency. Other related plots are available in the GitHub repository mentioned in Chapter 1.

4.2 Future work

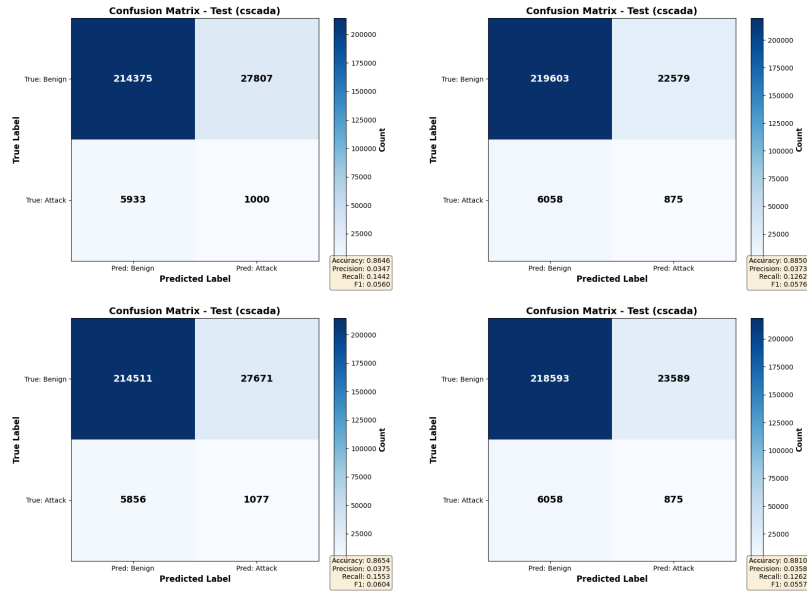
The current work is based on a network-flow-level representation of Modbus traffic, which limits the amount of information available to the model. As a result, the detection performance is not as strong as desired, especially for subtle or internally generated attacks. A more effective approach would involve using datasets that include process-level measurements (e.g., sensor readings, actuator states) or gaining access to packet payload information, which can provide deeper insight into the actual system behavior and control logic.

Once a decent model is built, future work can further explore lightweight anomaly detection models inspired by the Kitsune architecture and similar approaches, with the goal of developing solutions that are efficient enough for real-time deployment on resource-constrained devices such as embedded systems or platforms like a Raspberry Pi. Such models would be more practical for integration into real-world industrial environments, enabling continuous and on-device monitoring of CPS networks.

C-SCADA (LSTM approach): As discussed earlier, the multiple methods attempted resulted in the model predicting nearly all samples as benign, indicating poor learning due to class imbalance. Result for 80-20 train-test split is shown (same for all the three weighted models).



C-SCADA (Kit-NET-based approach): The following confusion matrices correspond to different hyperparameter settings. These show an improvement over the LSTM model, although performance remains limited.



External Attacker: The best performance is observed in this case, with higher detection capability compared to the compromised SCADA scenario. However, a noticeable number of false positives remain, which need to be addressed in future work.

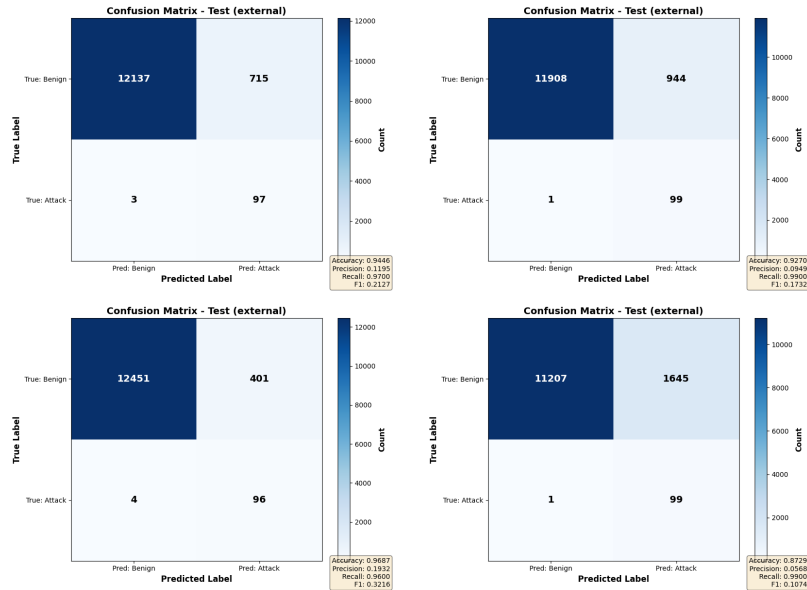


Figure 4.1: Confusion matrices across different models and scenarios

Acknowledgements

I express sincere gratitude to my guide, Prof. Chester Rebeiro, for providing me the opportunity to work on this project and for his overall support. I also thank Nikhilesh Singh, Post-Doctoral Research Associate, Technische Universität Darmstadt, for his initial guidance and suggestions, particularly in shaping the early direction of this work. A special thanks to Karthikeyan R, who mentored me during the later stages of the project. Despite joining close to the deadline, his guidance was extremely valuable in helping me better understand relevant research work and progress with the project. I am also grateful to my friends Shivanshu Kumar and Aadi Srivastava, for their support and for helping me with various doubts related to machine learning concepts throughout the course of this project. I further acknowledge the use of tools such as GitHub Copilot for assistance in code generation, and LLMs including Claude and ChatGPT for summarizing research papers and supporting the learning process. Finally, I would like to acknowledge all those who directly or indirectly contributed to the completion of this work.

Bibliography

- Abdelaty, M., R. Doriguzzi-Corin, and D. Siracusa (2022). “DAICS: A Deep Learning Solution for Anomaly Detection in Industrial Control Systems”. In: *IEEE Transactions on Emerging Topics in Computing* 10.2, pp. 1117–1129. DOI: [10.1109/TETC.2021.3073017](https://doi.org/10.1109/TETC.2021.3073017).
- Almendli, M. and J. B. Mohasefi (2024). “Anomaly detection system based on deep learning for cyber physical systems on sensory and network datasets”. In: *International Journal of Electrical and Computer Engineering (IJECE)* 14.6, pp. 6827–6837. ISSN: 2722-2578. DOI: [10.11591/ijece.v14i6.pp6827-6837](https://doi.org/10.11591/ijece.v14i6.pp6827-6837). URL: <https://ijece.iaescore.com/index.php/IJECE/article/view/36372>.
- Boakye-Boateng, K., A. A. Ghorbani, and A. H. Lashkari (2024). “Implementation of a Trust-Based Framework for Substation Defense in the Smart Grid”. In: *Smart Cities* 7.1, pp. 99–140. ISSN: 2624-6511. DOI: [10.3390/smartcities7010005](https://doi.org/10.3390/smartcities7010005). URL: <https://www.mdpi.com/2624-6511/7/1/5>.
- Canadian Institute for Cybersecurity (2023). *CIC Modbus Dataset 2023*. URL: <https://www.unb.ca/cic/datasets/modbus-2023.html>.
- Mirsky, Y. (2017). *KitNET-py: Python Implementation of KitNET*. <https://github.com/ymirsky/KitNET-py>. GitHub repository, accessed 2026-05-03.
- Mirsky, Y. et al. (Jan. 2018). “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection”. In: URL: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-3_Mirsky_paper.pdf.
- Sekaran, Y. et al. (2023). “Using Machine Learning to detect abnormalities on Modbus/TCP Networks”. In: *Proceedings of the 4th International Conference on Information Management & Machine Intelligence*. ICIMMI ’22. Jaipur, India: Association for Computing Machinery. ISBN: 9781450399937. DOI: [10.1145/3590837.3590893](https://doi.org/10.1145/3590837.3590893). URL: <https://doi.org/10.1145/3590837.3590893>.